

Architecture

Overview

BankOne is a **staff + customer-portal** core banking shell.

Backend code lives in `BankOne-BackEnd/` (Maven); UI in `BankOne-Frontend/` :

- **Backend:** Spring Boot 4.1, Java 21, packaged as WAR, deployed on **Open Liberty**
- **Frontend:** Angular SPA with JWT Bearer auth (staff `/app/`, portal `/portal/`)
- **Database:** PostgreSQL (`bankone`)
- **Notifications (implemented):** Kafka topic `bankone.notifications` → email (Mailpit locally, SendGrid HTTPS on Render)

Full technology and version inventory: [TECH_STACK.md](#).

Learning / platform roadmap (rate limit, cache, outbox, sharding lab): [TECH_LEARNING_PLAN.md](#).

```
flowchart LR
    UI[Angular SPA :4200] -->|HTTPS/HTTP JWT| API[Spring Boot on Liberty :9080]
    API --> PG[(PostgreSQL :5432)]
    API -->|publish BankActionEvent| Kafka[Kafka]
    Kafka --> Consumer[NotificationConsumer]
    Consumer --> Mail[Mailpit or SendGrid]
```

Package dependencies

```
flowchart TB
    subgraph backend [com.bankone]
        auth[auth]
        user[user]
        role[role]
        customer[customer]
        account[account]
        transaction[transaction]
        notification[notification]
        portal[portal]
        beneficiary[beneficiary]
        transfer[transfer]
        audit[audit]
        report[report]
        dashboard[dashboard]
        common[common]
    end
    auth --> user
    auth --> role
    auth --> audit
    customer --> account
    account --> customer
    account --> transaction
    account --> notification
    account --> audit
    portal --> account
    portal --> beneficiary
    portal --> transfer
    transfer --> account
    transfer --> audit
    report --> transaction
    report --> account
    report --> transfer
    user --> role
    dashboard --> customer
    dashboard --> account
    dashboard --> user
```

Implemented packages include `transaction`, `notification`, `portal`, `beneficiary`, `transfer`, `audit`, `report`. Still stub: `branch`, full `loan` product.

Layering (per feature module)

```
controller → service (interface + impl) → repository → entity
      ↓
      dto / enums / specification / util
```

Cross-cutting concerns live under `com.bankone.common`:

Area	Classes
Security	<code>SecurityConfig</code> , <code>JpaAuditingConfig</code>
Exceptions	<code>GlobalExceptionHandler</code> , domain exceptions
Audit base	<code>AuditableEntity</code>
Utils	<code>BusinessIdFormatter</code>

Frontend structure

```
src/app/
  core/          # auth, guards, interceptors, services, models, layout, config
  features/      # login, dashboard, customers, accounts, employees, profile, management
  app.routes.ts
```

Protected routes use `authGuard` with optional `data.roles`.

Deployment

Mode	Port	Notes
Open Liberty (primary)	9080 HTTP / 9443 HTTPS	<code>scripts/redeploy-liberty.sh</code> JDWP debug 7777 by default (<code>LIBERTY_DEBUG=0</code> to disable)
Kafka (local)	9092	<code>docker compose up -d kafka</code>
Mailpit (local)	SMTP 1025 / UI 8025	<code>docker compose up -d mailpit</code>
Embedded Tomcat	8080	<code>application.properties</code> <code>server.port</code>
Angular	4200	Talks to API base in <code>api-config.ts</code>

WAR artifact: `target/bankone-0.0.1-SNAPSHOT.war` → `$WLP_HOME/usr/servers/bankone/apps/`

Security architecture

1. Stateless JWT (`SessionCreationPolicy.STATELESS`)
2. Login public; all other endpoints authenticated
3. Role checks via `SecurityConfig` matchers (+ `@PreAuthorize` on some user APIs)
4. Roles loaded from DB on each request (`CustomUserDetailsService` → `UserRole`), **not** embedded in JWT claims
5. Account lockout via `LoginAttemptService` / user lock fields

Data architecture

- Schema evolution: Hibernate `ddl-auto=update`
- Explicit SQL: `schema.sql` creates `account_ordinal_seq` only
- Soft business IDs: `customerCode` is formatted in JSON (`BusinessIdFormatter`), not a DB column

Related docs

- [CALL_FLOW.md](#)
- [CLASS_DIAGRAM.md](#)
- [DATABASE_SCHEMA.md](#)